Mobile communications Mini Projects

Units 1: Lab Manual

Prepared September 2026

Table of Contents

Table of Contents 2
Min project 1.1: Simulating Link Budgets and Coverage Range with Python 3
Objective: 3
Tools and Software Required: 3
Background: 3
Procedure: 3
Expected Observations: 4
Lab Tasks: 4
Deliverable: 5
Mini Project 1.2: Comparing FDD and TDD Resource Utilization Under Traffic Asymmetry 5
Objective: 5
Problem Statement: 5
Requirements: 5
Suggested Approach: 5
Tools: 6
Deliverables: 6
Evaluation Criteria: 6
Mini Project 1.3: Validating and Extending the 0G-6G Generation Comparison 6
Objective: 6
Problem Statement: 6
Requirements: 6
Suggested Approach: 7
Tools: 7
Deliverables: 7
Evaluation Criteria: 7

Min project 1.1: Simulating Link Budgets and Coverage Range with Python

Objective:

Students will translate the link budget equation and free-space path loss formula from Section 1.10 into working Python code, then use that code to compare received power and maximum usable range across two frequency bands. By the end of the session, students will be able to compute, plot, and interpret P_RX versus distance, and will have a concrete numerical feel for why low-band spectrum reaches so much farther than millimeter-wave spectrum at equal transmit power.

Tools and Software Required:

- Python 3 (3.9 or later recommended)
- NumPy, for vectorized numerical computation
- Matplotlib, for plotting
- A Jupyter notebook (recommended, for inline plots) or a plain .py script run from a terminal
- No SDR hardware or radio equipment is required — this is a pure software simulation

Background:

Section 1.10 gives the link budget equation, P_RX = P_TX + G_TX − L_path + G_RX − L_other, with all quantities in dB or dBm. Given a transmitter's power and antenna gain, a receiver's antenna gain, and an estimate of the loss in between, this equation predicts the power available at the receiver's input, which must clear the receiver's sensitivity with margin to spare.

Path loss is estimated with the same simplified free-space formula used in the unit notes: L_path (dB) = 20·log10(d_km) + 20·log10(f_MHz) + 32.44. This makes explicit that path loss grows with the logarithm of both distance and frequency — so a fixed distance suffers substantially more loss at a much higher carrier frequency. Combining the two formulas lets a full P_RX-versus-distance curve be generated and compared against a sensitivity threshold to find maximum usable range.

Procedure:

1. Open a Jupyter notebook or a new file link_budget.py, and import numpy as np and matplotlib.pyplot as plt.
2. Write a function implementing free-space path loss, accepting distance in km and frequency in MHz and returning loss in dB. For example:

```
def fspl_db(d_km, f_mhz):
    return 20  np.log10(d_km) + 20  np.log10(f_mhz) + 32.44
```

3. Write a function implementing the link budget equation, accepting transmit power, transmit gain, path loss, receive gain, and other losses, and returning received power in dBm. For example:

```
def link_budget_dbm(p_tx_dbm, g_tx_db, l_path_db, g_rx_db, l_other_db):
    return p_tx_dbm + g_tx_db - l_path_db + g_rx_db - l_other_db
```

4. Define a NumPy array of distances on a log scale spanning short and long ranges, for example distances = np.logspace(-2, 2, 500), giving 500 points from 0.01 km to 100 km.
5. Define two frequency bands using the unit's worked examples: 700 MHz (low-band) and 28,000 MHz (mmWave).
6. Fix one consistent set of transmitter/receiver parameters for both bands so any difference in the resulting curves comes only from frequency — for example, 40 dBm transmit power, 15 dBi transmit antenna gain, 0 dBi receive antenna gain, and 3 dB of other losses.

7. For each band, compute path loss at every distance with the Step 2 function, then compute received power at every distance with the Step 3 function, producing two P_RX arrays of the same length as the distances array.
8. Choose a receiver sensitivity threshold, for example sensitivity_dbm = -100, consistent with the unit's worked examples.
9. Plot both P_RX curves against distance on one Matplotlib figure with a log x-axis (plt.xscale('log')), add a horizontal dashed line at the sensitivity threshold with plt.axhline, and label curves, axes, and a legend.
10. For each band, find the maximum distance at which P_RX still exceeds sensitivity using NumPy boolean indexing on the distances array, and compute the link margin (P_RX minus sensitivity) at a chosen reference distance.
11. Print a short summary listing, per band: frequency, path loss and P_RX at the reference distance, link margin, and maximum range above sensitivity.
12. Save the figure with plt.savefig('link_budget_comparison.png') and include it, the code, and the summary in the deliverable below.

Expected Observations:

- The 28 GHz curve falls off far more steeply than the 700 MHz curve as distance grows, since mmWave path loss accumulates faster per kilometer.
- With the fixed parameters above, the 700 MHz link should sustain a usable connection roughly one to two orders of magnitude farther than the 28 GHz link, echoing the coverage-versus-capacity trade-off in Section 1.5 and the two worked examples in Section 1.10.
- Both curves appear as straight lines on a log-distance axis, since the path loss formula is linear in log10(d_km) — a useful sanity check on the code.
- Computed maximum ranges should broadly match the qualitative reasoning in the unit notes: low-band reaching kilometers, mmWave reaching at most a few hundred meters under comparable gain assumptions.

Lab Tasks:

1. Raise the 28 GHz transmit antenna gain to 24 dBi (a phased-array value from Worked Example 1) while leaving 700 MHz unchanged, and quantify how much range this recovers.
2. Add a third band such as 3.5 GHz (the primary 5G NR mid-band) to the plot and describe where its curve falls relative to the other two.
3. Add an extra shadowing/fading margin (for example 8 dB) into L_other, and observe how much the maximum range shrinks for each band.
4. Using the computed P_RX values and an assumed fixed noise floor (for example -104 dBm for a 10 MHz channel), compute SNR versus distance and apply $C = B \cdot \log_2(1 + SNR)$ to estimate the capacity ceiling for a chosen bandwidth; comment on how quickly capacity degrades compared to P_RX.
5. Compare doubling transmit power versus doubling bandwidth in the Task 4 capacity calculation, and explain, using the logarithmic-versus-linear scaling from Section 1.10, why the two are not equally effective at raising data rate.
6. Without further simulation, write a short qualitative note on how TDD's guard periods and reduced per-direction slot time would change the effective transmit duty cycle assumed in a link budget compared to an FDD system transmitting continuously.

Deliverable:

Submit the complete Python source code, the saved plot of P_RX versus distance for both bands with sensitivity line and legend, and a short written analysis (roughly half a page) explaining the observed range difference in terms of the free-space path loss formula.

Mini Project 1.2: Comparing FDD and TDD Resource Utilization Under Traffic Asymmetry

Objective:

Students will build a simplified simulator modeling how FDD and TDD allocate uplink and downlink capacity over time, and use it to quantify how each scheme's effective throughput and utilization respond to asymmetric, downlink-heavy traffic, connecting the qualitative discussion in Section 1.8 to concrete numbers.

Problem Statement:

Section 1.8 explains that FDD reserves fixed, symmetric capacity on two paired channels, while TDD shares one channel by dividing it into time slots — at the cost of guard periods and network synchronization, but with the ability to rebalance the uplink-to-downlink slot ratio to match demand. Real mobile traffic is heavily downlink-dominated. This project asks: for a fixed total capacity and a given degree of traffic asymmetry, how much usable capacity does each scheme actually deliver, and how does that change as asymmetry grows?

Requirements:

- Model FDD as two independent, fixed-capacity channels (uplink and downlink), each continuously available with equal, fixed bandwidth per direction.
- Model TDD as one channel of the same total bandwidth as an FDD pair, divided into a repeating frame with a configurable downlink/uplink slot split and a configurable guard-period overhead that carries no traffic.
- Accept a traffic demand scenario as a downlink-to-uplink data volume ratio (for example 80:20).
- Compute, for each scheme, effective throughput per direction and overall utilization (useful traffic delivered as a fraction of total capacity available) across at least three asymmetry scenarios (for example 50:50, 70:30, 90:10).

Suggested Approach:

1. Define fixed parameters: total bandwidth, the resulting fixed per-direction FDD capacity, slots per TDD frame, and a guard-period overhead (for example one guard slot in twenty).
2. Implement FDD's effective throughput and utilization for a given demand ratio, noting that its fixed per-direction capacity cannot be reallocated, so spare capacity in an under-demanded direction goes unused.
3. Implement TDD's effective throughput, letting the downlink/uplink slot ratio be set per scenario to approximately match demand, subject to guard-period overhead reducing total usable slots.
4. Run both models across the required asymmetry scenarios and tabulate or plot utilization and throughput for FDD versus TDD in each.
5. Rerun the TDD calculation with at least two different guard-period fractions to show how larger switching overhead erodes TDD's flexibility advantage.
6. Summarize in writing the asymmetry level beyond which TDD's rebalancing outweighs guard-period overhead, and the range where FDD remains competitive.

7. Optionally, extend the model to a multi-frame time series to visualize slot allocation over several TDD frames against FDD's continuous parallel channels.

Tools:

- Python 3 with NumPy for calculations
- Matplotlib for comparison charts
- A Jupyter notebook is recommended for inspecting results scenario by scenario

Deliverables:

- Complete, runnable simulator code for both the FDD and TDD models
- A comparison chart or table of effective throughput and utilization for FDD versus TDD across all required scenarios
- A one- to two-page written report referencing the FDD/TDD trade-offs from Section 1.8 (paired-spectrum requirement, synchronization needs, guard-period overhead, slot-ratio flexibility) and stating when TDD's advantage is largest

Evaluation Criteria:

- Correctness of the FDD and TDD utilization/throughput models, including proper guard-period accounting: 30 percent
- Code quality, structure, and reproducibility: 20 percent
- Depth of analysis across asymmetry and guard-period scenarios, with well-reasoned conclusions: 30 percent
- Presentation, including chart/table clarity and report quality: 20 percent

Mini Project 1.3: Validating and Extending the 0G-6G Generation Comparison

Objective:

Students will critically research and validate the generation comparison table in Section 1.2 against additional current sources, then produce either a descriptive infographic design or a small rule-based Python recommender tool mapping a stated communication problem to the generation that first solved it.

Problem Statement:

The Section 1.2 summary table compresses seven decades of history into a few rows of access technology, primary service, and approximate peak data rate. Such tables are useful for quick reference but can obscure nuance, regional variation, and the fact that headline peak rates are rarely achieved in practice. This project asks students to verify the table's claims against additional sources, identify points of nuance it omits, and build a small artifact demonstrating which generation solved which class of problem.

Requirements:

- Cross-check at least four of the six generation rows (0G/1G through 6G) against at least two additional credible sources per generation, noting discrepancies, especially around peak-versus-typical data rates.
- Identify, in writing, at least two specific problems that motivated moving from one generation to the next, beyond the table itself (for example, 1G's fragmentation and security weaknesses that motivated GSM, or circuit-switched data's inefficiency that motivated GPRS, both discussed in the unit notes).
- Produce one of: (a) a written and sketched infographic design summarizing 0G-6G, specifying layout, the claims each section displays, and its intended audience; or (b) a Python program that takes a plain-

language description of a communication requirement (for example, "low-latency control for a robotic arm") and returns, via simple keyword matching, which generation first addressed that class of problem and why.
- Produce a short written report tying the research and chosen deliverable together.

Suggested Approach:

1. List every factual claim in the Section 1.2 table (access technology, primary service, peak data rate per generation) as a checklist to verify.
2. Research each generation using at least two independent sources beyond the unit notes, recording sources and any numbers that differ, with a brief explanation of why (for example, theoretical peak versus field-measured throughput).
3. Draft specific problems solved by each generational transition, drawing on the unit's narrative on 1G fragmentation/security, GPRS/EDGE proving packet switching, and the three 5G service categories (eMBB, URLLC, mMTC).
4. Choose between the infographic-description and recommender-tool deliverables, and sketch its structure first: panel layout and anchoring data points for the infographic, or the keyword/generation mapping for the tool.
5. Build the chosen deliverable — either the structured infographic description, or the recommender tool as a Python script using basic string matching (a dictionary mapping keywords to a generation and justification is sufficient; no machine learning required).
6. If building the tool, test it against at least six varied example problem statements, including one that does not map cleanly to a single generation, and record the results.
7. Write the final report tying verified research to the chosen deliverable, discussing at least one case where real-world data nuanced the unit's simplified table.

Tools:

- General research resources (standards body publications, reputable technical references)
- Python 3, only if the recommender-tool option is chosen; standard-library string handling is sufficient
- Any drawing tool the student already has, only if actually sketching the infographic rather than describing it in prose

Deliverables:

- A short annotated fact-check of the Section 1.2 table, with sources consulted and discrepancies found
- A written list of at least two problems-per-transition motivating each generational shift, beyond the table itself
- Either the infographic design description or the recommender tool's source code with test cases and outputs
- A final one- to two-page report synthesizing the research and explaining how the chosen deliverable communicates it

Evaluation Criteria:

- Accuracy and thoroughness of research validation, including credible sourcing: 30 percent
- Quality and clarity of the identified generational problems/motivations: 20 percent
- Quality of the chosen deliverable (infographic coherence, or recommender tool correctness and test coverage): 30 percent
- Presentation and writing quality of the final report: 20 percent